

Towards an FPGA-Based Dedicated Computer for Molecular Dynamics Simulations

2.0

Peter Hamm

*Department of Chemistry, University of Zurich,
Winterthurerstrasse 190, CH-8057 Zurich,
Switzerland corresponding author: peter.hamm@chem.uzh.ch*

(Dated: April 20, 2026)

I. INTRODUCTION

The present document describes mostly the technical aspects of the 2nd version of an FPGA-based dedicated computer for Molecular Dynamics (MD) simulations, following Ref. [1]. The implementation now includes all features needed to run MD simulations of proteins, while Ref. [1] implemented only Lennard Jones interactions and focused on the networking aspect. Some of the text in the following chapters has been duplicated from Ref. [1], to make it a self-contained document, while most parts are new or have been updated significantly.

II. HARDWARE

Each node of the MD cluster is realized with inexpensive development boards AX7201 from ALINX (<https://www.en.alinx.com/>). It features what is considered a mid-ranged FPGA, Artix 7 XC7A200T-2FGG484I from Xilinx/AMD, together with – very important for the present project – four bidirectional optical interfaces running at 4 Gb/s each and four 1 Gb/s ethernet interfaces. The optical interfaces as well as two of the ethernet interfaces are used to connect nodes in a 3D torus, as discussed further below, and a third ethernet interface for communication with the host computer. It also contains an expansion port with 34 freely programmable in/out (IO) lines used to set the node number, to synchronize all nodes and to control a cooling fan for the FPGA. To that end, a small additional printed-circuit board (PCB) has been designed. Finally, 1 GB of external DDR3 SDRAM is available on the development board, which stores all force field parameters of the bonded interactions. The other interfaces of the development board (a USB and a VGA port) are not used.

The Artix 7 XC7A200T FPGA contains 33650 logical slices, each containing four look-up-tables (LUTs) with six inputs as well as eight flip-flops. It furthermore contains 730 dual port block RAMs (BRAMs) with 18Kb each, and 740 signed 25 bit×18 bit-multipliers. Much of the design of the MD algorithm is directly related to the way how the BRAMs and multipliers can be configured, for an efficient as possible usage of the resources of the FPGA. The FPGA is programmed in Verilog with the Vivado development tool, whose licence is free of charge for the Artix 7 XC7A200T chip.

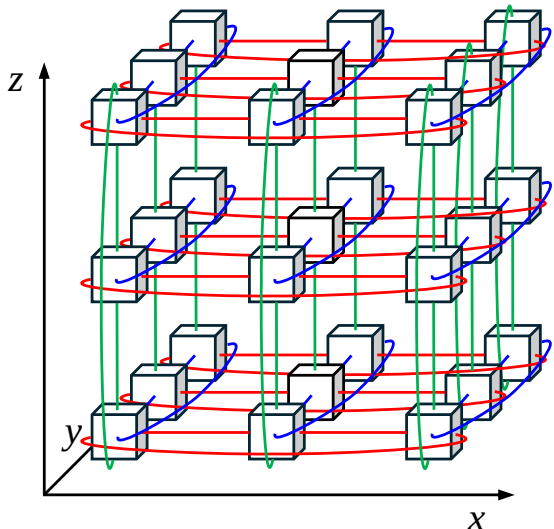


FIG. 1. A $3 \times 3 \times 3$ configuration with 27 nodes connected as a 3D torus. Links in the x -direction are shown in red, those in the y -direction in blue, and those in the z -direction in green. Each link is bidirectional. Not all cyclic links in the z -direction are shown in order not to overcrowd the picture.

III. DATA FLOW

To understand the following, one needs to carefully distinguish a “node” from a “box”. Each node corresponds to one evaluation board with one FPGA. Each node can house only 256 atoms. For larger MD simulations, a certain number of nodes need to be connected and run in parallel. Each node contains a “home box”, which houses the atoms of the particular node. However, for the calculation of the interaction of the atoms in the home box with other atoms, which may be in a neighboring node, also the positions of the atoms in the neighboring nodes need to be known. For computation time reasons, one cannot afford to request these positions from neighboring nodes at the moment when they are needed for the calculation; despite the reasonably high speed of the interfaces that would be way too slow. This is why there are 26 additional boxes in a given node, which contain the data of the neighboring nodes; they will be called “neighboring boxes”. Whenever the MD algorithm finished to calculate the new positions and velocities of one

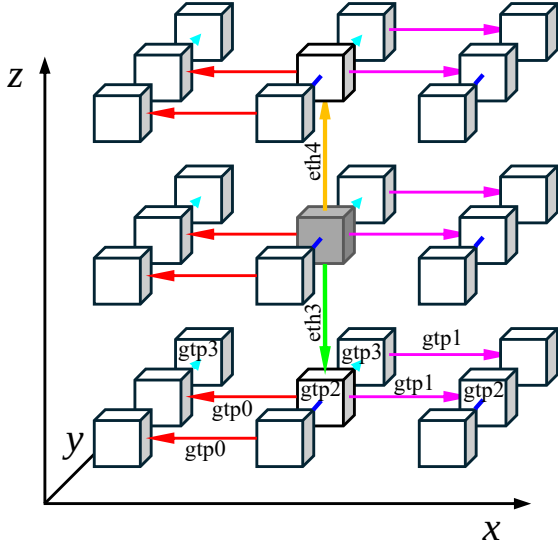


FIG. 2. Organisation of the link between a central node (in grey) and its 26 neighboring nodes via two ethernet interfaces (eth3 and eth4) and four optical interfaces (gtp0-gtp3). See text for details.

atom, its data are broadcasted to all neighboring nodes, and stored into one of its neighboring boxes, so that the information is available at high speed, i.e., one for each clock cycle, for the force calculation in the next MD time step.

A. Linking the Nodes

The nodes are connected to reveal a 3D torus to account for the periodic boundary conditions, see Fig. 1. The FPGA is programmed in a way that the size of the overall MD system can be chosen freely with only minimal adjustments needed. A $3 \times 3 \times 3$ configuration is the smallest meaningful configuration, in which case the 26 neighboring boxes around the home box of a given node contain the same information as the home boxes of the 26 neighboring nodes. In a larger configuration, a given node will no longer “see” all other nodes. The number of nodes in the various directions does not have to be identical, allowing one to adjust the overall system to the shape of a particular MD problem.

Each node is linked to its 26 neighboring nodes via two ethernet interfaces (eth3 and eth4) and four optical interfaces (gtp0-gtp3), see Fig. 2. The organisation of these links accounts for the fact that the optical interfaces are faster (4 Gb/s) than the ethernet interfaces (1 Gb/s). That is, the two ethernet interfaces connect only up and down along the z -directions, and the receiving nodes funnel the data through into their (xy) -planes. Communication within the (xy) -planes is performed by the four optical interfaces, which connect up and down in the x -

and y -directions, respectively. To also reach the corners of the (xy) -planes, each node in the $\pm x$ -direction or the $\pm y$ -direction also funnels through the atom data set by one additional step perpendicular to the previous one, e.g. the node in the $(+x)$ -direction funnels the atom data through to the $(+x, -y)$ -node, etc. For a given atom, each ethernet interface thus needs to transmit only one data set, while each optical interface needs to transmit six data sets (i.e., three (xy) -planes at $-z$, $z = 0$ and $+z$ times two data transfer steps in the (xy) -planes). Given that the overhead of an ethernet frame is larger than that of a frame of the optical interface, the transmission speeds of both types of interface are well balanced. Together with its frame, the transmission of one atom data set, i.e., 27 bytes (see Fig. 5 below) to all 26 neighboring nodes takes 480 ns. If more than about 100 atoms are in a node, this is faster than the force calculation, and hence not speed-limiting for the overall performance of the MD computer. Data integrity is guaranteed by a 3 byte CRC and a 1 byte sequence number.

B. Synchronisation

Synchronisation of nodes is only needed after the calculation of all atoms, i.e., once per MD time step. To that end, a separate link is used that is realized via the freely programmable IO lines the FPGA development board provides. The link connects a busy signal of all nodes via an open-collector OR logic function. Only when all nodes have finished their calculation of a particular MD time step, and broadcasted all results, they proceed with the calculation of the next MD time step. If a transmission error between nodes should occur (which never happened), the corresponding MD step is repeated. To that end, an error signal is transmitted to all other nodes in the same way as the synchronisation signal.

C. Data Flow Within Each Node

Fig. 3 shows the overall data flow within each node. The central MD machine consists of the home box and the 26 neighboring boxes (of which only 8 in the (xy) -plane are shown in Fig. 3 in order not to overcrowd the picture). The atoms in the neighboring boxes may be within the cut-off distance to the home box, and hence need to be considered in the force calculation. Initially, the host computer writes atom data into the home box (it does so for all nodes). The data flow through a multiplexer (MUX) in front of the home box, since other data may enter into the home box as well, such as the initial data sent from the host computer.

Each MD time step produces new atom data, which are written back into the home box via the MUX, but only after checking whether their (xyz) -coordinates are still within the range of the home box, as some atoms might have moved out. In addition, the newly generated

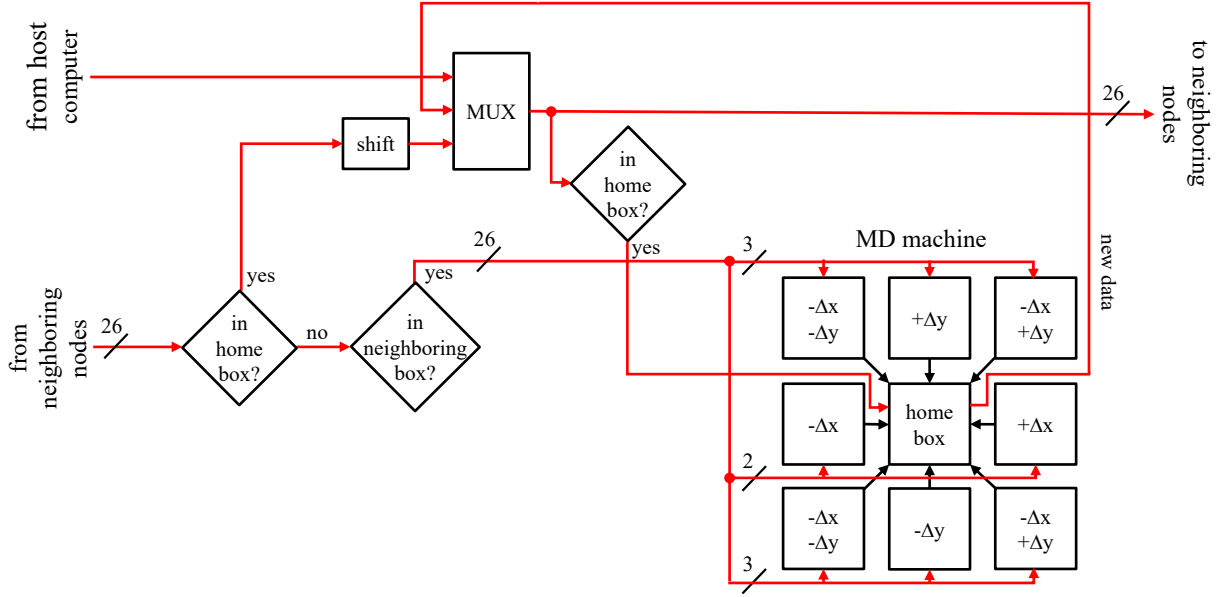


FIG. 3. Overall data flow within each node, see text for details. Of the in total 26 neighboring boxes around the home box, only 8 in the (xy) -plane are shown in order not to overcrowd the picture. Red arrows indicate data flow, black arrows force calculations (as discussed in Sec. IV and summarized in Fig. 6).

data are broadcasted to all neighboring nodes, regardless whether they are still within the range of the home box. All other nodes will do the same, hence, their results appear at the receiving side of Fig. 3. If the (xyz) -coordinates of the received data moved into the range of the home box of the node considered here (and hence moved out of the home box of the corresponding neighboring node), they will enter the home box via the MUX. In that way, an atom moved from the home box of a neighboring node into the home box of the node considered here. On the other hand, if the (xyz) -coordinates of the received data did not move into the range of the home box of the node considered here, but rather are in the range of the corresponding neighboring box, they are loaded into it.

D. Organization of the Data RAMs

Atom data in the home box and all neighboring boxes are stored twice in two types of RAMs, one indexed by their position in sub-boxes, the second indexed by their atom number. The first is needed for the calculation of the non-bonded interactions (see Sec. IV B 3), the second for the bonded interactions (see Sec. IV C 4). In either case, the RAM size is twice as large as actually needed for the amount of atom data to be stored. During calculation, data are read from one half and the results are written in parallel into the other half, making use of the fact that the RAMs have two ports with simultaneous writing and reading access. From one MD time step to the next one, the role of these two half's

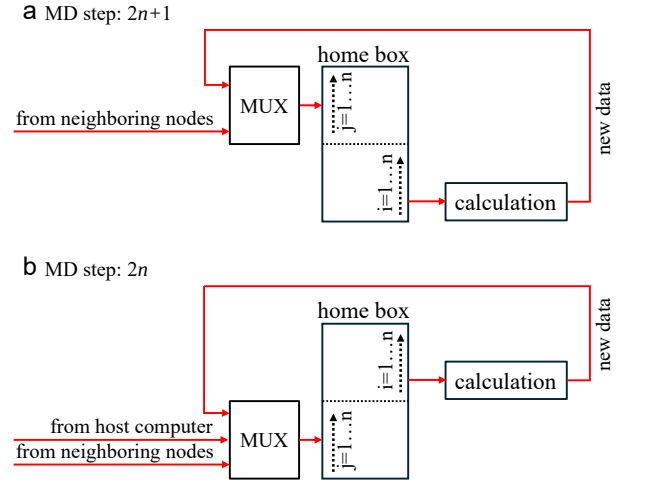


FIG. 4. Organisation of the RAM holding the home box data, and they way they are written and written for an odd MD step (panel a) and an even MD step (panel b). The MUX is the same as in Fig. 3, while the yes-no flowcharts of Fig. 3 are not shown here in order not to overcrowd the picture. The concept is exemplified here for the home box; the neighboring boxes are organized in a similar manner.

is swapped. Upon initialisation from the host computer, data are written into the lower block (Fig. 4b), and then evaluated in the first MD step (Fig. 4a). The sequence continues from there on. The number of MD steps always should be an even number. Furthermore, the MD algorithm is organized such that new data of a given atom are fully calculated before treating the next atom. In

	velocities			meta data							positions		
	v_z	v_y	v_x	route	resv.	constr.- type	m	q	LJ- type	atom Nr.	z	y	x
bit:	24	24	24	2r+6	13r	3	6	18	1r+5	2r+16	24	24	24

FIG. 5. Structure of the atom data, containing the velocities v_x , v_y and v_z and positions x , y , and z . The meta data in between contain the atom-number, Lennard Jones-type, charge q , mass m , constraint type, and the information into which box an atom is supposed to be routed to. The second row shows the number of bits used for the corresponding data, where “r” stands for reserve (i.e., currently not used).

that way, while the calculation of further atoms is still ongoing, the data of the already finished atoms can be broadcasted to the neighboring nodes, thereby causing only minimal time overhead for the communication between nodes, provided that the transmission of the data for each atom is faster than its calculation.

The RAM indexed by sub-box position, used for the calculation of the non-bonded interactions, can hold a maximum of 256 atoms for each of the home box or the neighboring boxes. A fatal error signal is generated when the number of atoms per node exceeds an threshold of 252, which aborts the calculation. It was found that this never happens when the average number of atoms per nodes is $\lesssim 220$. The RAM indexed by atom number, used for the calculation of the non-bonded interactions, can hold in total 4096 atoms in the current implementation (see also Sec. IV C 4).

Fig. 5 shows the structure of the full atom data consisting of 27 byte, whose various entries will be discussed below. Not all data are needed in all cases, e.g. the BRAMs containing the atoms in neighboring boxes need to store only the positions x , y , z , the atom number, Lennard Jones-type and the charge q for the calculation of the non-bonded interactions with the atoms in the home box. To save resources, their bit-size is reduced accordingly.

IV. MD ALGORITHM

Fig. 6 shows a flow chart of the central MD-machine that performs the calculations of one MD time step within each node. An outer loop over all atoms in the home box - we will call atom i_1 in the following - initiates in total 11 parallel pipelines for the calculation of the non-bonded and bonded forces. Each of these pipelines loops over the atoms i_2 , with which atom i_1 interacts via the particular force for which the pipeline is in charge of. Once all forces $F_{i_1,j}$ are ready, they are summed up, and the velocities and positions of atom i_1 are updated, potentially applying a thermostat, performing a center-of-mass motion correction and constraining some of the bond distances by Shake. The new data are then broadcasted among all nodes, in the way shown in Fig. 3. The various aspect of the MD-machine are discussed in the

following.

A. Time Propagation and Units

Newtons equation of motion are time-propagated with a Leapfrog algorithm:

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \frac{\mathbf{F}_i}{m_i} \Delta t \quad (1)$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \Delta t \quad (2)$$

where $\mathbf{x}_i$ is the position of atom i , $\mathbf{v}_i$ its velocity, m_i its mass, $\mathbf{F}_i$ the force acting on it and Δt the time step. The left arrow indicates how $\mathbf{v}_i$ and subsequently $\mathbf{x}_i$ are updated in each MD time step. In order to minimize the computations that need to be done within the FPGA to the absolute minimum, we wish to write Eqs. 1 and 2 as:

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \frac{1}{m_i} \mathbf{F}_i \quad (3)$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \quad (4)$$

This hides the time step in the forces, and forces as well as velocities have the same unit as positions. The mass of an atom m_i is defined relative to a reference mass m_{ref} , which was chosen to be 16 (as that simplifies the way the inverse of mass m_i can be tabularized in a LUT). The reference mass m_{ref} is also hidden in the force calculation, while the mass m_i of a given atom is part of the meta data (see Fig. 5). Furthermore, we define positions in units of the cut-off distance r_{cut} , which will be the size of the home box. Hence, all properties in Eqs. 3 and 4 are in fact unit-less.

B. Non-Bonded Interactions

1. Lennard Jones Interactions

A prototype Lennard-Jones potential is defined as:

$$V_{LJ} = 4\epsilon_{ref} \left(\frac{\sigma_{ref}^{12}}{r_{ij}^{12}} - \frac{\sigma_{ref}^6}{r_{ij}^6} \right) \quad (5)$$

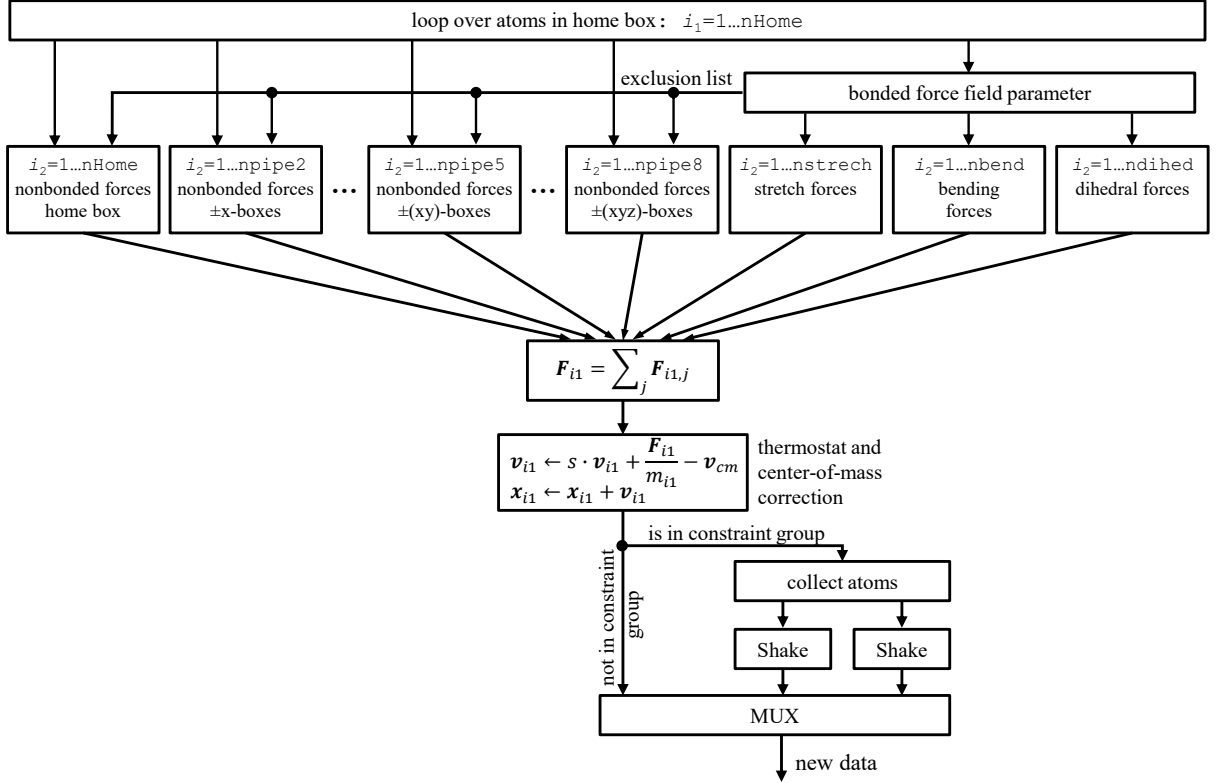


FIG. 6. Organisation of the central MD machine with in total 11 parallel pipelines performing the calculations of the non-bonded forces as well as stretch, bend and dihedral forces. Besides the pipeline that calculate the non-bonded forces of the $\pm x$ -boxes and the $\pm(xy)$ -boxes, both of which shown in the figure, there are additional pipelines for the $\pm y$ -boxes, $\pm z$ -boxes, $\pm(xz)$ -boxes and $\pm(yz)$ -boxes, respectively. The results of all pipelines are added up to calculate the force $\mathbf{F}_{i1}$ on atom i_1 . Subsequently, new velocities $\mathbf{v}_{i1}$ and positions $\mathbf{x}_{i1}$ are calculated, taking into account also a scaling factor s from the thermostat and correction of the center-of-mass motion (see Eq. 43). If atom i_1 is in not in a constraint group, the calculation is finished at this point, otherwise, Shake is applied to constrain a certain number of bond lengths to the desired value. Since Shake works on more than one atom, the newly calculated atom data need to be stored in an intermediate storage until all atoms of a constraint group appear in the stream of atoms i_1 . Two Shake pipelines work in parallel to match the calculation speed with that of the force calculation. Each connection in that flow chart contains a FIFO, to leverage out changing calculation times of the various force pipelines. The different aspects of this figure are discussed in more detail in the text.

where ϵ_{ref} is a reference Lennard Jones binding energy, σ_{ref} a reference Lennard Jones distance, and r_{ij} the distance between atoms i and j . The force acting on atom i is:

$$\mathbf{F}_i = -\frac{\partial V_{LJ}}{\partial \mathbf{x}_i} = 24\epsilon_{ref} \left(2\frac{\sigma_{ref}^{12}}{r_{ij}^{14}} - \frac{\sigma_{ref}^6}{r_{ij}^8} \right) (\mathbf{x}_i - \mathbf{x}_j) \quad (6)$$

The prototype Lennard-Jones potential Eq. 6 with ϵ_{ref} and σ_{ref} is programmed into a LUT, see Sec. IV G. To be able to consider pairs of different atom types during the simulation, r_{ij}^2 is scaled with:

$$r_{ij}^2 \leftarrow \frac{\sigma_{ref}^2}{\sigma_{ij}^2} r_{ij}^2 \quad (7)$$

before entering the calculation of Eq. 6, and the final force with

$$\mathbf{F}_i \leftarrow \frac{\epsilon_{ij}}{\epsilon_{ref}} \frac{\sigma_{ref}}{\sigma_{ij}} \mathbf{F}_i \quad (8)$$

afterwards. The scaling factors are tabularized in another LUT for different pairs of atom. To that end, we define a “Lennard Jones-type” of each atom, which is contained in the meta data (see Fig. 5), and apply the usual combination rules:

$$\sigma_{ij} = \frac{\sigma_i + \sigma_j}{2} \quad (9)$$

and

$$\epsilon_{ij} = \sqrt{\epsilon_i \epsilon_j} \quad (10)$$

The Lennard-Jones binding energies ϵ_{ref} are typically given in units of kJ/mol. Multiplying that with:

$$\epsilon_{ref} \leftarrow \epsilon_{ref} \cdot \frac{10^{-6} dt^2}{m_{ref} r_{cut}^2} \quad (11)$$

reveals the desired unit-less Lennard-Jones binding energy. Here, dt is in units of fs, r_{cut} the cut-off distance in

units of nm, and m_{ref} an reference mass in atomic mass units. Furthermore, when the Lennard-Jones distance r_{ref} is written in units of r_{cut} , r_{ij} and $\mathbf{x}_i$ are as well.

2. Coulomb Interactions

We write the Coulomb interaction in the following form:

$$V_{\text{Coul}} = \frac{e^2}{4\pi\epsilon_0 r_{\text{cut}}} \frac{q_1 q_2}{r_{ij}} \quad (12)$$

where e is the elementary charge, and ϵ_0 the vacuum permittivity. In this expression, the charges q_1 and q_2 of the two interacting atoms are unit-less, and the distance r_{ij} as well. The corresponding force is:

$$\mathbf{F}_i = -\frac{\partial V_{\text{Coul}}}{\partial \mathbf{x}_i} = \frac{e^2}{4\pi\epsilon_0 r_{\text{cut}}} \frac{q_1 q_2}{r_{ij}^3} (\mathbf{x}_i - \mathbf{x}_j) \quad (13)$$

Furthermore, for the time being we consider the reaction field model for the Coulomb interaction, which effectively switches the resulting force to zero at the cut-off distance, provided that the dielectric constant is $\gtrsim 20$ [2]:

$$\mathbf{F}_i = \frac{e^2}{4\pi\epsilon_0 r_{\text{cut}}} q_1 q_2 \left(\frac{1}{r_{ij}^3} - 1 \right) (\mathbf{x}_i - \mathbf{x}_j) \quad (14)$$

The charge q of each atom is contained in the meta data, see Fig. 5.

The pre-factor $e^2/4\pi\epsilon_0 r_{\text{cut}}$ has units of $\text{kg}\cdot\text{m}^2/\text{s}^2$. When multiplying it with:

$$\frac{e^2}{4\pi\epsilon_0 r_{\text{cut}}} \leftarrow \frac{e^2}{4\pi\epsilon_0 r_{\text{cut}}} \cdot \frac{10^3 N_A dt^2}{m_{\text{ref}} r_{\text{cut}}^2} \quad (15)$$

with N_A the Avogadro constant, the desired unit-less Coulomb energy is obtained.

3. Nearest Neighbor Search

Non-bonded forces are calculated only for atoms that are within a certain cut-off distance, typically around 1 nm. To determine which atoms are within that cut-off distance around a given atom, it is common in MD programs to first calculate a nearest neighbor list. While this requires only the calculation of a distance, and thus is less expensive than the full force calculation, it still requires on the order of n^2 such calculations (when no additional tricks are used such as those discussed below), where n is the number of atoms.

Calculating such a nearest neighbor list is against the very concept of the FPGA algorithm. That is, the concept of the FPGA algorithm is to fully pipeline the force calculation, hence, it takes a single clock cycle with a certain latency for the calculation of the force between pairwise two atoms, see Fig. 8 below. The calculation

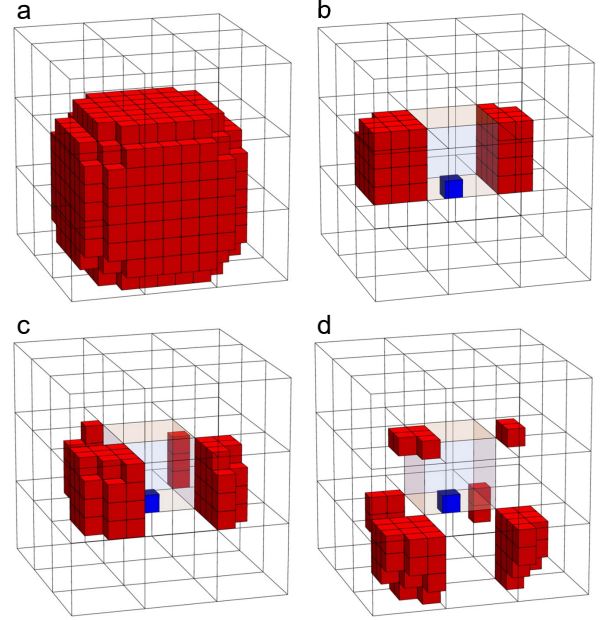


FIG. 7. (a) The sub-boxes within cut-off distance, when using $4 \times 4 \times 4 = 64$ sub-boxes. The embedding 27 boxes are shown as well. (b-d) Parallelizing the calculation of non-bonded forces between one atom in the home box and another atom in one of the neighboring boxes. Sub-boxes in the neighboring boxes considered by the different parallel pipelines are shown in red, and the sub-box in the home box, in which the one atom resides, is shown in blue. The home box (in faint white), as well as the neighboring boxes, are also indicated. Panel (b) exemplifies the sub-boxes in the $\pm x$ -direction, and two additional ones exist for the $\pm y$ and $\pm z$ -directions. Panel (c) exemplifies the sub-boxes in the (xy) -plane, and two additional ones exist for the (xz) and (yz) -planes. Finally, panel (d) shows the sub-boxes in the (xyz) -cube corners.

of the nearest neighbor list cannot be faster than that; just the latency would be shorter since the calculation is simpler. Precalculating a nearest neighbor list would not save any computation time.

The size of the home box is set to one cut-off distance, hence the trivial solution to the nearest neighbor problem would be to just consider all atoms within the home box and its 26 surrounding neighboring boxes. While this would safely include all atom pairs that are within the cut-off distance, very many are separated by a larger distance. For these pairs, one would know without calculation that the force is zero, but it would still require one clock cycle to loop over it. To estimate the waste of clock cycles in that case, we consider the ratio of volume of a sphere with a radius equal to the cut-off distance versus that of the 27 boxes [3]:

$$f_V = \frac{4\pi/3 r_{\text{cut}}^3}{27 r_{\text{cut}}^3} = 0.16 \quad (16)$$

84% of the clock cycles would be wasted in that case.

To improve on that, each box is divided into $4 \times 4 \times$

4 = 64 sub-boxes, a common strategy also in conventional MD programs. When new atom data are received, they are sorted into their corresponding sub-box and are stored in a RAM that is indexed by that sub-box number. No n^2 -algorithm to generate a nearest neighbor list is needed for that. When choosing sub-boxes in a way that all atom pairs within the cut-off distance are captured, as shown in Fig. 7a, the ratio f_V increases almost three fold to ≈ 0.44 . The ratio would increase even further if more sub-boxes were used (it converges very slowly to 1), but then it would become likely that some sub-boxes contain zero atoms. This again would result in wasted clock cycles, since one has to check each sub-box whether or not it contains atoms. $4 \times 4 \times 4 = 64$ sub-boxes was deemed to be the best compromise.

In total 8 parallel pipelines perform the calculation of the non-bonded interactions, see Fig. 6. That is, one pipeline works on atom pairs with both atoms in the home box. For a given atom i_1 , it needs to consider all 64 sub-boxes of the home box. Three pipelines work on the sub-boxes in the two boxes in the $\pm x$, $\pm y$ and $\pm z$ -directions, respectively (Fig. 7b exemplifies that for the $\pm x$ -direction). Depending on the sub-box in the home box in which atom i_1 resides (shown in blue in Fig. 7b for one example), these are either 80 or 78 sub-boxes in the two neighboring boxes, in average 79.5 sub-boxes. Three further pipelines work on the corners in the $\pm(xy)$, $\pm(xz)$ and $\pm(yz)$ -planes (exemplified for the (xy) -plane in Fig. 7c) with 81 sub-boxes in average. Finally, the 8th pipeline works on the (xyz) -cube corners (Fig. 7d) with 67.5 sub-boxes in average). The rather complex structure of the sub-boxes that need to be considered (see Fig. 7b-d) is programmed into a LUT. The computational effort of the pipelines of the types shown in Figs. 7b and 7c, which do the biggest share of the calculations since it is three of them each, is extremely well balanced.

4. Exclusion of Non-Bonded Interactions

MD force fields typically exclude the calculation of Lennard Jones and Coulomb interactions for atoms that are one or two chemical bonds apart, and reduce their interaction energy by a fudging factor for atoms that are three chemical bonds apart. To that end, the pipelines for the non-bonded interactions need to know which atoms i_2 around a currently calculated atom i_1 are within two or three chemical bonds, respectively. That exclusion list would be too large to be included into the meta-data of an atom, hence it is stored in the memory that also stores all force field parameters for the bonded interaction, see Sec. IV C 4. A pipeline is placed in front of all force pipelines, reading and distribution the exclusion list to the non-bonded pipelines, together with the force field parameters for bonded pipelines, see Fig. 6

C. Bonded Interactions

1. Bond Stretch

The harmonic potential between two bonded atoms is written as:

$$V_{stretch} = \frac{k_{ij}}{2} \left(|d\mathbf{x}_{ij}| - r_{ij}^{(0)} \right)^2 \quad (17)$$

where $d\mathbf{x}_{ij} = \mathbf{x}_i - \mathbf{x}_k$ is the vector connecting both atoms, k_{ij} the force constant and $r_{ij}^{(0)}$ the equilibrium distance. The corresponding force is:

$$\mathbf{F}_i = -\frac{\partial V_{stretch}}{\partial \mathbf{x}_i} = k_{ij} \left(\frac{r_{ij}^{(0)}}{|d\mathbf{x}_{ij}|} - 1 \right) d\mathbf{x}_{ij} \quad (18)$$

The force constants are typically tabulated in units of kJ/mol/nm², hence when scaling them with

$$k_{ij} \leftarrow k_{ij} \cdot \frac{10^{-6} dt^2}{m_{ref}} \quad (19)$$

the same unit-less representation as discussed for the non-bonded interactions is obtained.

2. Bond Bend

The bend potential is written as:

$$V_{bend} = \frac{k_{ijk}}{2} \left(\phi_{ijk} - \phi_{ijk}^{(0)} \right)^2 \quad (20)$$

with

$$\cos \phi = \frac{d\mathbf{x}_{ij} \cdot d\mathbf{x}_{kj}}{|d\mathbf{x}_{ij}| |d\mathbf{x}_{kj}|}. \quad (21)$$

Here, $d\mathbf{x}_{ij} = \mathbf{x}_i - \mathbf{x}_j$ and $d\mathbf{x}_{kj} = \mathbf{x}_k - \mathbf{x}_j$, and $\mathbf{x}_j$ is the position of the middle atom. To obtain the force, the chain rule has to be applied various times [4]:

$$\mathbf{F}_i = -\frac{V_{bend}}{\mathbf{x}_i} = k_{ijk} \left(\phi_{ijk} - \phi_{ijk}^{(0)} \right) \frac{1}{\sin \phi} \cdot \left(\frac{1}{|d\mathbf{x}_{ij}| |d\mathbf{x}_{kj}|} d\mathbf{x}_{kj} - \frac{d\mathbf{x}_{ij} \cdot d\mathbf{x}_{kj}}{|d\mathbf{x}_{ij}|^3 |d\mathbf{x}_{kj}|} d\mathbf{x}_{ij} \right) \quad (22)$$

and along the same lines for $\mathbf{F}_k$. For the middle atom, we find $\mathbf{F}_j = -\mathbf{F}_i - \mathbf{F}_k$.

When the bending force constants are given in units of kJ/mol/rad², they need to be scaled with

$$k_{ijk} \leftarrow k_{ijk} \cdot \frac{10^{-6} dt^2}{m_{ref} r_{cut}^2} \quad (23)$$

to obtain the same unit-less representation as discussed for the non-bonded interactions.

3. Bond Dihedrals

At the basis of the dihedral potential is a Fourier series

$$V_{dihed} = \frac{1}{2} \sum_{m=1}^3 k_{ijkl}^{(m)} \left[1 + \cos \left(m \cdot \phi_{ijkl} - \delta_{ijkl}^{(m)} \right) \right] \quad (24)$$

with

$$\cos(\phi_{ijkl}) = \frac{\mathbf{n}_{ijk} \cdot \mathbf{n}_{jkl}}{|\mathbf{n}_{ijk}| |\mathbf{n}_{jkl}|}. \quad (25)$$

and

$$\begin{aligned} \mathbf{n}_{ijk} &= d\mathbf{x}_{ij} \times d\mathbf{x}_{kj} \\ \mathbf{n}_{jkl} &= d\mathbf{x}_{kj} \times d\mathbf{x}_{kl} \end{aligned} \quad (26)$$

with $d\mathbf{x}_{ij} = \mathbf{x}_i - \mathbf{x}_j$, $d\mathbf{x}_{kj} = \mathbf{x}_k - \mathbf{x}_j$ and $d\mathbf{x}_{kl} = \mathbf{x}_k - \mathbf{x}_l$. In Ref. 5, this is called the “cross product definition” of the dihedral angle ϕ_{ijkl} . Proper and improper dihedrals are calculated with the same potential energy form.

We will only consider force fields with phase shifts $\delta_{ijkl}^{(m)}$ being either 0 or π (such as the Amber force fields), in which case Eq. 24 can be recast in the form (that special form is used to simplify the derivate Eq. 32 as much as possible):

$$V_{dihed} = - \sum_{m=1}^3 \frac{A_{ijkl}^{(m)}}{m} \cos^m(\phi_{ijkl}) \quad (27)$$

with

$$\begin{aligned} A_{ijkl}^{(1)} &= \frac{3}{2} k_{ijkl}'^{(3)} - \frac{1}{2} k_{ijkl}'^{(1)} \\ A_{ijkl}^{(2)} &= -2 k_{ijkl}'^{(2)} \\ A_{ijkl}^{(3)} &= -6 k_{ijkl}'^{(3)} \end{aligned} \quad (28)$$

where $k_{ijkl}'^{(m)} = k_{ijkl}^{(m)}$ when the phase shift is $\delta_{ijkl}^{(m)} = 0$, and $k_{ijkl}'^{(m)} = -k_{ijkl}^{(m)}$ when it is $\delta_{ijkl}^{(m)} = \pi$. We discard the constant factor $A_{ijkl}^{(0)}$, since that does not cause any force.

With that, one obtains for the force, if it is an outer atom i [4–6]:

$$\mathbf{F}_i = - \frac{\partial V_{dihed}}{\partial \phi_{ijkl}} \cdot \frac{|d\mathbf{x}_{kj}|}{|\mathbf{n}_{ijk}|^2} \cdot \mathbf{n}_{ijk} \quad (29)$$

and if it is an inner atoms j :

$$\mathbf{F}_j = \left(\frac{d\mathbf{x}_{ij} \cdot d\mathbf{x}_{kj}}{|d\mathbf{x}_{kj}|^2} - 1 \right) \mathbf{F}_i - \frac{d\mathbf{x}_{kl} \cdot d\mathbf{x}_{kj}}{|d\mathbf{x}_{kj}|^2} \mathbf{F}_l \quad (30)$$

which includes the force on the other outer atom l

$$\mathbf{F}_l = + \frac{\partial V_{dihed}}{\partial \phi_{ijkl}} \cdot \frac{|d\mathbf{x}_{kj}|}{|\mathbf{n}_{jkl}|^2} \cdot \mathbf{n}_{jkl} \quad (31)$$

Here, we need

$$\frac{\partial V_{dihed}}{\partial \phi_{ijkl}} = \sum_{m=1}^3 A_{ijkl}^{(m)} \sin(\phi_{ijkl}) \cos^{m-1}(\phi_{ijkl}) \quad (32)$$

and:

$$\sin(\phi_{ijkl}) = \frac{|d\mathbf{x}_{kj}| (d\mathbf{x}_{ij} \cdot \mathbf{n}_{jkl} + d\mathbf{x}_{kl} \cdot \mathbf{n}_{ijk})}{2 |\mathbf{n}_{ijk}| |\mathbf{n}_{jkl}|}. \quad (33)$$

The two terms, $d\mathbf{x}_{ij} \cdot \mathbf{n}_{jkl}$ and $d\mathbf{x}_{kl} \cdot \mathbf{n}_{ijk}$ are identical in exact arithmetics, but are both calculated for a symmetric expression even in the presence of round-off errors.

The force on other inner atom k is not calculated explicitly, since forces are calculated only on one atom at a time, as it appears along the pipeline shown in Fig.6 (i.e, it will be teated when atom k appears in the pipeline).

When the bending force constants are given in units of kJ/mol, they need to be scaled with

$$k_{ijkl} \leftarrow k_{ijkl} \cdot \frac{10^{-6} dt^2}{m_{ref} r_{cut}^2} \quad (34)$$

to obtain the same unit-less representation as discussed for the non-bonded interactions.

4. RAMs for Atom Data and Force Field Parameters

From an organisational point of view, bonded interactions are different from non-bonded interactions in two regards. First, the amount of force parameters needed for a given atom i_1 (about 1 kB) would be way to large to include them into the meta data. The force field parameters of all atoms are therefore stored locally in each individual node, since it is *per se* not known which atom will be in which node. The BRAMs inside the FPGA would be too small for that, hence the external DDR3 RAM of the evaluation board is used. The particular DDR3 RAM can be read with a speed of 3.2 GB/s (minus some overhead), hence it takes about 350 ns to read the force field data for each atom i_1 , as it appears in the pipeline of Fig. 6. If there are more than about 100 atoms per node, that is faster than the force calculation and thus is not limiting the overall calculation speed.

Second, the (xyz) -coordinates of the atoms i_2 that interact with a given atom i_1 need to be indexed by atom number, since this is how bonded forces are defined, and not by sub-box number as for the non-bonded interactions. The atoms i_2 can either be in the home box or in one of the neighboring boxes. As new atoms appear in a node after each individual MD time step, they are stored in a RAM indexed by atom number. Since again, one does not know which atom will be in which node, the RAM must be large enough to store all atoms of a given MD setup, not only those in the home box or the neighboring boxes. One set of (xyz) -coordinates will be needed per each clock cycle for the calculation, and the external DDR3 RAM is too slow for that. Therefore, internal BRAMs are used. However, BRAMs are a potentially limiting resource of the FPGA, and they need to be used in a very efficient manner. To that end, the total space of atom numbers is divided into “protein atoms” (which may also include small molecule ligands,

etc.) and those for water molecules and single-atom ions. Only the typically much smaller number of protein atoms are stored in that BRAM, which is currently limited to 4096 entries (a parameter that can be easily increased when migrating to a larger FPGA). Single-atom ions do not require any bonded force field calculation, and water molecules are treated separately, see next Sec. IV D.

One pipeline each works on the stretch, bend and dihedral forces, see Fig. 6. To save resources (in particular multipliers), and since the bonded force calculations will not be the speed limiting aspect, these pipelines perform one force calculation for every three clock cycle only (with a certain latency). Nonetheless, to avoid that the calculations of the three types of forces interfere with each other, the (xyz)-coordinates of the atoms i_2 are triplicated into three BRAMs, which can all be read in parallel and independently from each other.

D. Constraining XH Bond Distances

Water molecules result in the by far largest number of atoms in a typical MD setup. Their stretch and bend forces are not calculated, rather the OH and HH-distances are constraint to a fixed value, as is common practice in MD simulations. Besides the limitation of RAM space discussed above, constraining the OH-distance avoids the high frequency vibration of the OH bond, allowing for a larger time-step of typically 2 fs, or even 4 fs for D₂O. The same applies to O-H, C-H, N-H and S-H bonds among the protein-atoms (even though their stretch forces are calculated, see above).

For one pair of atoms, there is an analytical expression to achieve that, typically referred to as SHAKE and/or RATTLE [7]. That analytical expression involves a quadratic equations whose solution requires a square-root, which would be resource-intensive to implement in an FPGA. However, when the distances of more than one atom-pair are to be constrained, in particular when constraining the structure of a water molecule, the problem is typically solved iteratively. That is, one cycles through the distances of pairwise two atoms. Since the correcting force for one pair affects that of the other ones, one has to iterate that procedure a couple of times. In the following, an algorithm avoiding the square-root will be discussed that is iterative already for a single pair of atoms. However, since one has to do it iteratively anyways when more than one atom pair is constraint, and that is the most common situation, it turns out one can just as well use that less resource-intensive algorithm. It has been tested that it converges to the same result as the conventional SHAKE algorithm [7], and requires only an insignificantly larger number of iterations (around 20%).

The distance r of each pair of atoms after an MD time step is supposed to be constraint to a fixed distance r_0 . While the calculation of r requires a square-root, r^2 can easily be calculated in an FPGA:

$$r^2 = |\mathbf{x}_i + \mathbf{v}_i - \mathbf{x}_j - \mathbf{v}_j|^2. \quad (35)$$

Here, $\mathbf{x}_{i,j}$ are the positions of the atoms before updating them according to Eq. 4, and $\mathbf{v}_{i,j}$ the new velocities after adding the forces according to Eq. 3. We relate r^2 to r_0^2 :

$$r'^2 = \frac{r^2}{r_0^2}. \quad (36)$$

Since r_0 is a constant that does not vary during the MD simulation, $1/r_0^2$ can be precalculated in the host computer, and the division can be replaced by a multiplication. The goal is to converge $r'^2 - 1$ to zero. We have $r'^2 - 1 = (r' - 1)(r' + 1) \approx 2(r' - 1)$, since r' will not deviate much from 1 during the MD simulation. We thus find:

$$\Delta r' \equiv r' - 1 \approx \frac{r'^2 - 1}{2} \quad (37)$$

where the division by 2 is a simple shift operation. With that, the velocities are updated:

$$\mathbf{v}_i \leftarrow \mathbf{v}_i - \frac{m_j}{m_i + m_j} \Delta r' (\mathbf{x}_i - \mathbf{x}_j) \quad (38)$$

and equivalently for $\mathbf{v}_j$. From here on, one iterates by going back to Eq. 35 until convergence is achieved. Just like conventional SHAKE [7], the algorithm conserves the center-of-mass motion and the angular momentum.

A group of atoms XH_n , whose XH-distances are to be constraint, is called a “constraint group”, where X is a heavy atom (C, N, O or S) with n H’s bonded to it. “Constraint-type” in the meta data (see Fig. 5 provides the information for the Shake pipelines whether it is a water molecule (in which case the two OH bonds and the HH-distance are constraint), or how many XH bonds in a XH_n group are to be constraint. Shake needs the positions and velocities of all atoms within a constraint group. When atom data are distributed over the network of nodes is not fully predictable, since other nodes might interfere with distributing their own data, and hence the atoms of a constraint group don’t necessarily appear subsequently in the stream of atoms i_1 . Atom data therefore need to be stored in an intermediate storage until all atoms of a constraint group have appeared. Shake is then initiated on the whole constraint group. Two Shake pipelines work in parallel to match the calculation time with that of the force calculation.

For this to work, it is required that all atoms of a given constraint group are in the same node/box. To that end, the various H’s of a constraint group are sent to the same node/box as the one heavy atom of that constraint group, even if their coordinates are slightly outside of that node/box. “Route” in the meta data (see Fig. 5 determines into which node an atom is sent.

E. Thermostat

A simple thermostat is implemented using velocity rescaling, taking into account the constraints of an FPGA

for performing certain arithmetic operations and the architecture of the interconnected nodes. First, the temperature $T = 1/(3Nk_B) \sum_i m_i \cdot v_i^2$ is calculated in the unit-less representation discussed above:

$$T = \sum_i m_i v_i^2 \quad (39)$$

where the sum runs over all atoms of the home box and its 26 neighboring boxes. The contribution of the home box is calculated after each MD step, and then broadcasted to all neighboring nodes, hence, the temperature T is available with one time step delay. Since the thermostat has a time constant that is multiple times the time step, that delay is not a problem. The temperature is referenced to a target temperature:

$$r = \frac{T}{T_{target}} \quad (40)$$

Since divisions are slow and resource-intensive in an FPGA, the inverse of T_{target} is calculated in the host computer in the same unit-less representation and then multiplied with T using one of the dedicated multipliers of the FPGA. When defining a velocity rescaling factor

$$s = r^{-\frac{1}{2n_\tau}}, \quad (41)$$

a thermostat with time constant $\tau = n_\tau \cdot dt$ is obtained, where dt is the MD time step. That expression is calculated with the following trick: The temperature ratio r will be a number $r = 1 + r'$ with $r' \ll 1$, since the temperature never will deviate much from the target temperature. With that, we can power-expand the scaling factor as:

$$s = 1 - \frac{r'}{2n_\tau} \quad (42)$$

Only powers of 2 are considered for n_τ , in which case the division in Eq. 42 can be performed by an inexpensive shift operation. Finally, Eq. 3 is modified by introducing the velocity rescaling factor s for each MD time step:

$$\mathbf{v}_i \leftarrow s \cdot \mathbf{v}_i - \mathbf{v}_{cm}/n'_\tau + \frac{1}{m_i} \mathbf{F}_i \quad (43)$$

For large enough n_τ and small r' , the power expansion Eq. 42 reveals in leading order the same scaling factor as the Berendsen thermostat [8], and the known deviations from a correct canonical ensemble are considered to be acceptable for the time being [9]. A stochastic term for the velocity rescaling thermostat to ensure a correct kinetic energy distribution [10] will be possible without significant impact on the simulation speed and will be added in a future version.

In addition, the center of mass motion, which accumulates due to round-off errors, is subtracted from the velocities for each time step with a time-constant

$n'_\tau dt \approx 2dt$ [11]. The center of mass motion is calculated as

$$\mathbf{v}_{cm} = \frac{1}{m_{tot}} \sum_i m_i \mathbf{v}_i \quad (44)$$

where m_{tot} is the total mass. The center of mass motion is averaged over all atoms of the home box and the 26 neighboring boxes in the same way as the temperature.

The center-of-mass motion of the atoms within every node as well as their contribution to the total kinetic energy are broadcasted over the whole network once the calculation of all atoms i_1 is finished. On the receiving side, they are added up and the new scaling factor s and total center-of-mass velocities are ready before the calculation of the new MD time step starts. The current state of the thermostat and center-of-mass correction are readable and writable by the host computer, allowing for a bit-exact restart of a MD simulation.

F. Fixed Point Arithmetics

All calculations are performed in fixed-point arithmetics, with bit-sizes that are optimized for efficient usage of in particular the BRAMs used to store atom data, and the multipliers, both of which are potentially limiting resources of an FPGA. The impact of fixed-point arithmetic for MD simulations has thoroughly been studied [12]. The MD problem is in fact particularly well suited for a fixed-point representation, since we know the range of positions and velocities quite well. That is, the position of all atoms is obviously bound to the MD box (recall the positions are defined in units of the box-size). Velocities are Gaussian distributed:

$$\rho_k \propto e^{-\frac{m_i v_k^2}{2k_B T}} \quad (45)$$

with $k \in \{x, y, z\}$. The width of the Gaussian distribution scales with $\sqrt{1/m_i}$ and $\sqrt{T}$, respectively, i.e. varies relatively little, and the probability goes to zero very quickly beyond that width. Should an overflow occur, a fatal error is generated and the MD simulation is stopped (which never happened unless the initial condition of an MD setup were such that it explodes). Finally, for the calculations of the bonded forces, with comparable complex intermediate results, we can utilize the fact that chemical bonds have bond distances in relatively small range between about 1 Å and 2 Å.

For the positions, we choose a fixed-point representation from -2...+2 in 24 bit, to be able to represent differences of positions between two atoms, one of which is in the home box and the other in one of the neighboring boxes. For velocities, we choose a fixed-point representation from $-1/2^s$ to $+1/2^s$ in 24 bit, where $s=4$ is chosen so that the probability of an overflow is negligible, considering Eq. 45. When evaluating the sum in Eq. 4, the lower $s-1$ bits are disregarded, and the subsequent bit is

used for round off. Since the velocities accumulate forces over many MD time steps (Eq. 3), it is important to represent them at higher bit resolution than actually needed for Eq. 4, to minimize round-off errors during accumulation. Finally, forces are also represented from $-1/2^s$ to $+1/2^s$, albeit in 32 bit. Forces also accumulate the contributions from many atoms i_2 , hence it is also important represent them at higher precision than what is needed for the sum in Eq. 3.

To put these bit-sizes into relation, it should be noted that the significand of a single-precision floating-point number also has 24 bits, and that common MD programs typically do use single precision floating-point arithmetics only. We will see that the energy stability, which is a critical test of round-off errors, is excellent, despite (or maybe because of) the fixed-point arithmetics.

G. Function LUT's

While adders are relatively easy to implement in an FPGA with its universal logic slices, and dedicated multipliers exist, divisions, as well as any more complicated function such as square-root, etc., would be very resource-expensive. Whenever possible, they are avoided, e.g., the division by σ_{ij}^2 in Eq. 7 is replaced by a multiplication with $1/\sigma_{ij}^2$, where the latter can be pre-calculated in the host computer (that has de done only once before the MD simulation is started).

When that is not possible, anything more complicated than additions and multiplications, from divisions up to the first terms Lennard Jones and Coloumb forces Eqs. 6 and 14, respectively, is performed by quadratic interpolation with the help of a LUT. That is common practice in FPGA implementations [3, 13–18]. To that end, the bit-range of the input data into a given function f is separated into two parts. The upper part (in the current implementation, 9 bit) is used to address a particular entry of LUT, while the lower part (17 bit) is interpreted as a number $x \in (0, 1)$. The following is then calculated:

$$f = A_0 + A_1 \cdot x + A_2 \cdot x^2 \quad (46)$$

with properly precalculated coefficients A_0 , A_1 and A_2 that are tabularized in the LUT. Eq. 46 contains additions and multiplications only. Function evaluation is fully pipelined with one calculation per clock-cycle and a latency of 6 clock clock cycles.

For example, the first terms of Eq. 6 as a function of r_{ij}^2 are calculated in that way, while the multiplication with $(\mathbf{x}_i - \mathbf{x}_j)$ is performed explicitly within the FPGA. The same applies to e.g. the arccos- and $1/\sin$ -functions in Eq. 22.

V. PARALLELIZATION AND PIPELINING

FPGAs offer extreme design flexibility enabling massive parallelization and very dedicated pipelining, with-

out the overhead of shared data busses. This is how FPGA implementations can outperform GPUs in terms of simulation speed, despite the fact that FPGAs can not compete GPUs in terms of simple benchmark numbers such as the number of arithmetic operations per second. Currently, GPUs are the state-of-the-art in MD simulations [19, 20].

In the current design, parallelization happens on 3 levels. First, many nodes run in parallel with each node being in charge only for a relatively small number of atoms. For an efficient exchange of data, the way how nodes are interlinked must represents the topology of the problem, a 3D-torus (see Figs. 1 and 2).

The second level happens inside the FPGA by parallelizing in total 11 pipelines for the force calculation, see Fig. 6, 8 of which are for the calculation of the non-bonded forces (see discussion in Sec. IV B 3), and one each for the stretch, bend and dihedral forces. The calculation of the non-bonded forces, is the limiting aspect in the overall simulation speed, while stretch, bend and dihedral forces require a much smaller number of calculations.

In that regard, it is important to mention that each connecting data line in the flow chart of Fig. 6 contains a FIFO as an intermediate memory, which stores data when they are ready, and delivers them when they are needed. These FIFOs very efficiently average out fluctuating calculation times. It is also important to mention that the FIFOs are constructed from BRAMs and LUT-RAMs that are distributed throughout the FPDA chip, hence the routing software implementing a design onto the chip can really physically place them between the block that produces the data and the block that further processes them. This makes data flow inside the chip very efficient, despite the fact that some of the data lines in Fig. 6 are very wide, more than 200 bits.

The third level of parallelization happens inside every one of the force pipelines. To illustrate that, Fig. 8 shows the pipeline used for the non-bonded interactions. Anything that can be calculated in parallel is in fact calculated in parallel, in particular the Lennard Jones forces and the Coulomb forces, but also e.g., dx^2 , dy^2 and dz^2 needed to calculate r_{ij}^2 as an input for the force calculations. The overhead that distributing data for a parallelization on such a fine level would generate in a conventional computer architecture would be prohibitive. In an FPGA implementation, in contrast, each line in the flow chart of Fig. 8 is routed as physical data line on the chip.

Pipelining, in turn, happens on two levels. Returning to the non-bonded pipeline of Fig. 8, each dashed horizontal line represents one clock cycle. While it takes 28 clock cycles for a force calculation, which is what is called the “latency” of pipeline, it is “fully pipelined”. That is, one set of input data can be pushed into the pipeline every single clock cycle, and one result per clock cycle will eventually appear at the output. Also the larger boxes in Fig. 8, which extend over more than one clock cycle, e.g., the multipliers or the force calculations, are fully

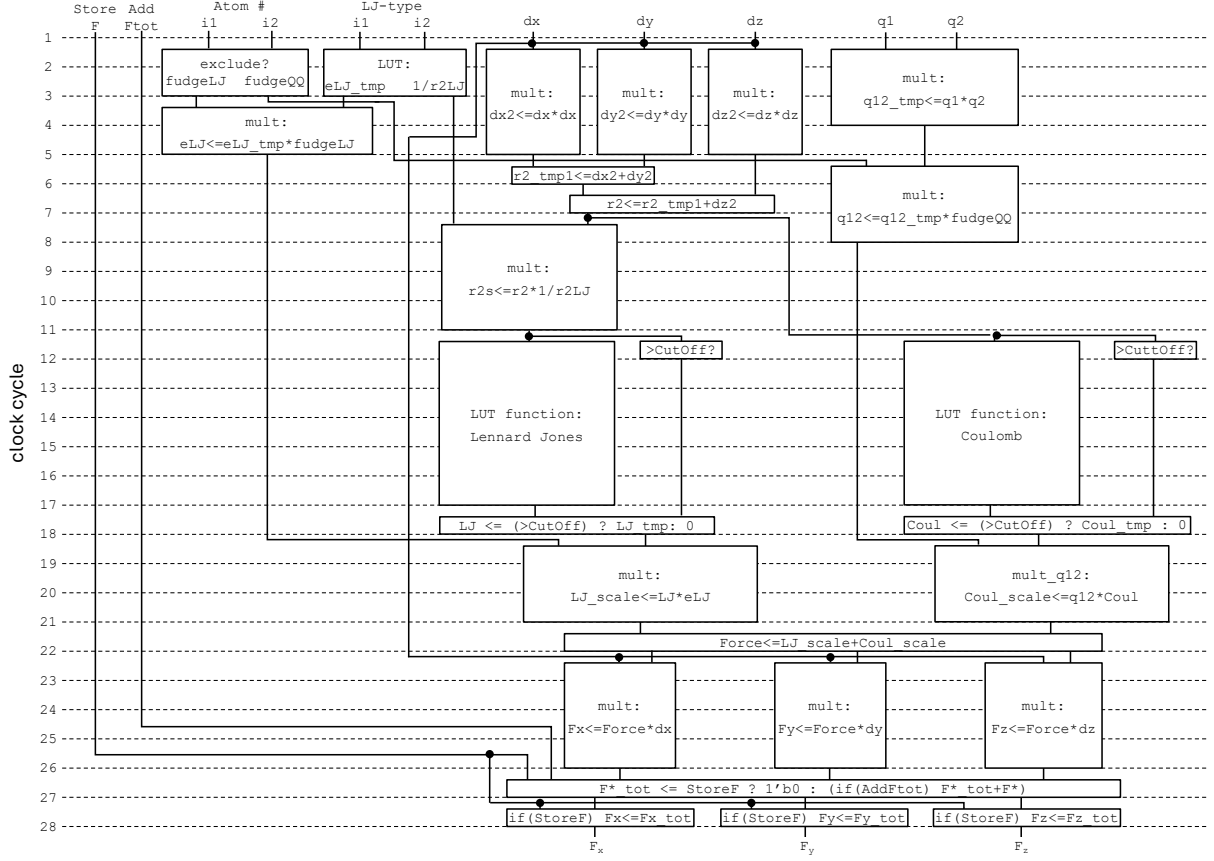


FIG. 8. Pipeline for the calculation of the non-bonded interactions Eqs. 6 and 14. One pairwise interaction is calculated per clock cycle with a latency of 28 clock cycles. It starts from $dx = x_1 - x_2$, $dy = y_1 - y_2$ and $dz = z_1 - z_2$, as well as the physical properties determining the interaction, i.e., the LJ-types of atoms i_1 and i_2 for the Lennard Jones parameters, the charges q_1 and q_2 as well as atom numbers of atoms i_1 and i_2 to determine whether they are within 2 or 3 chemical bonds. It outputs accumulated forces F_x , F_y and F_z . The dashed horizontal lines indicate clock cycles in the pipeline and boxes the operations performed at a particular step of the pipeline. If a signal has to be transferred for more than one clock-cycle, it is delayed by the proper amount of clock cycles with the help of a shift register.

pipelined internally. While modern CPUs, of course, perform pipelining as well, they are designed to do that in a versatile manner. Dedicated pipelining as the one shown in Fig. 8, and that intermixed with parallelization, would not be possible.

On a larger level, the whole MD-machine is pipelined, see Fig. 6. For example, while Shake is applied to atoms for which the forces have already been calculated, the forces on the next atom are already been calculated. Also broadcasting new data among neighboring nodes is pipelined in this way. For this to work, it has been a strict design premise to calculate the forces on one atom i_1 at a time. This comes at the expense that force calculations that include n atoms, e.g., $n = 2$ for the non-bonded interactions, have to be performed n times. To illustrate that, let's again consider the non-bonded interactions. It is common in MD programs that each atom pair is considered only once by making use of Newton's third law of motion, calculating the force between them and then having this force acting on both atoms with

opposite signs. In terms of nested loops over atoms, the outer loop i_1 runs from 1 to n (where n is the number of atoms), and the inner loop from 1 to $(i_1 - 1)$, hence $n(n-1)/2$ force calculations. In the present case, it would be reasonably straight forward to do that for atom pairs that are within the home box. However, for atom pairs, in which one atom is in one of the neighboring nodes, which amounts to about 88% of the force calculations, that would result in unacceptably large data traffic over the links connecting the nodes. Therefore, both inner and outer loops run from 1 to n , increasing the computational effort by a factor 2. Performing redundant calculations to avoid communication is a strategy that has been used before [21, 22].

VI. CONCLUSION

The current implementation includes all features needed to perform MD simulations of proteins. In de-



FIG. 9. Snapshot of an MD simulation of the Villin Headpiece, performed in the FPGA cluster. A short movie of the same MD simulation can be found on GitHub (<https://github.com/pehammUZH/Towards-an-FPGA-Based-Dedicated-Computer-for-Molecular-Dynamics-Simulations>)

tail, it features:

- Arbitrary size of the MD setup, in principle. Increasing the number of nodes, and thereby the number of atoms that can be simulated, is only a matter of money, but does not affect the simulation speed.
- Lennard Jones interactions with the usual cut-off.
- Coloumb interactions, at this point with effectively a cut-off by using a reaction field model [2].
- Bonded interactions, i.e. stretch, bend and di-hedral forces. The software on the host computer reads Gromacs topology files and translates them

into what is needed to write the force field parameters into the FPGA nodes.

- Simple velocity scaling thermostat for NVT calculations (barostat still missing).
- Shake to constrain XH bonds.

The simulation speed, around 1 μ s/day with 2 fs time steps, is the same as that in Ref. [1], since the calculation of the non-bonded forces is still the speed-limiting aspect. All new features have been added in a way that it does not slow down the calculation, by either pipelining or parallelizing them. Overall, the required FPGA resources about doubled in comparison to Ref. [1], but are still below 50% of the resources the current FPGA, Artix 7 XC7A200T, provides. There is sufficient room to complement the list of features.

Fig. 9 shows a snapshot from an MD simulation if the villin headpiece HP35 performed in the FPGA cluster. The villin headpiece HP35 is a very small model protein with 35 amino acids that has served in the past as a test case to validate the performance of MD simulations [23, 24]. We currently do so as well in ongoing work.

Acknowledgement: I wish to thank Markus Meuwly for insightful discussions on the project and Yoshitaka Tanimuara for his hospitality during my sabbatical stay at Kyoto University, where this project started. The research has been supported financially by the University of Zurich

Data Availability Statement: The Verilog code of the present MD implementation, the C-code running on the host computer to control the FPGA cluster, as well as the design of the PCB board connecting the IO lines of the development board are deposited on GitHub: <https://github.com/pehammUZH/Towards-an-FPGA-Based-Dedicated-Computer-for-Molecular-Dynamics-Simulations>.

-
- [1] P. Hamm, Toward an FPGA-based dedicated computer for molecular dynamics simulations, *J. Chem. Phys.* **162**, 054108 (2025).
- [2] O. Steinhauser, Reaction Field Simulation of Water, *Mol. Phys.* **45**, 335 (1982).
- [3] C. Yang, T. Geng, T. Wang, R. Patel, Q. Xiong, A. Sanaullah, C. Wu, J. Sheng, C. Lin, V. Sachdeva, W. Sherman, and M. Herbordt, Fully integrated FPGA molecular dynamics simulations, SC '19 Proc. Int. Conf. High Perform. Comput. Networking, Storage Anal. 2019; , 1 (2019).
- [4] R. C. van Schaik, H. J. C. Berendsen, A. E. Torda, and W. F. van Gunsteren, A Structure Refinement Method Based on Molecular Dynamics in Four Spatial Dimensions, *J. Mol. Biol.* **234**, 751 (1993).
- [5] H. Bekker, H. J. C. Berendsen, and W. F. van Gunsteren, Force and Virial of Torsional-Angle-Dependent Potentials, *J. Comput. Chem.* **16**, 527 (1995).
- [6] A. Blondel and M. Karplus, New Formulation for Derivatives of Torsion Angles and Improper Torsion Angles in Molecular Mechanics : Elimination of Singularities, *J. Comput. Chem.* **17** (1996).
- [7] A. R. Leach, *Molecular Modelling: Principles and Applications* (Pearson Education Limited, Essex, 2001).
- [8] H. J. C. Berendsen, J. P. M. Postma, W. F. van Gunsteren, A. DiNola, and J. R. Haak, Molecular dynamics with coupling to an external bath, *J Chem Phys* **81**, 3684 (1984).

- [9] T. Morishita, Fluctuation formulas in molecular-dynamics simulations with the weak coupling heat bath, *J. Chem. Phys.* **113**, 2976 (2000).
- [10] G. Bussi, D. Donadio, and M. Parrinello, Canonical sampling through velocity rescaling, *J. Chem. Phys.* **126**, 014101 (2007).
- [11] S. C. Harvey, R. K.-Z. Tan, and T. E. Cheatham, The flying ice cube: Velocity rescaling in molecular dynamics leads to violation of energy equipartition, *J. Comput. Chem.* **19**, 726 (1998).
- [12] S. Le Grand, A. W. Götz, and R. C. Walker, Spfp: Speed without compromise—a mixed precision model for gpu accelerated molecular dynamics simulations, *Computer Physics Communications* **184**, 374 (2013).
- [13] N. Azizi, I. Kuon, A. Egier, A. Darabiha, and P. Chow, Reconfigurable molecular dynamics simulator, 12th Annu. IEEE Symp. Field-Programmable Cust. Comput. Mach. Napa, CA, USA, , 197 (2004).
- [14] Y. Gu, T. VanCourt, and M. C. Herbordt, Accelerating molecular dynamics simulations with configurable circuits, *Int. Conf. F. Program. Log. Appl.* 2005, Tampere, Finl. , 475 (2005).
- [15] R. Scrofano, M. Gokhale, F. Trouw, and V. K. Prasannat, A hardware/software approach to molecular dynamics on reconfigurable computers, 2006 14th Annu. IEEE Symp. Field-Programmable Cust. Comput. Mach. Napa, CA, USA, , 23 (2006).
- [16] V. Kindratenko and D. Pointer, A case study in porting a production scientific supercomputing application to a reconfigurable computer, 4th Annu. IEEE Symp. Field-Programmable Cust. Comput. Mach. Napa, CA, USA , 13 (2006).
- [17] Y. Gu, T. Vancourt, and M. C. Herbordt, Improved interpolation and system integration for FPGA-based molecular dynamics simulations, 2006 Int. Conf. F. Program. Log. Appl. Madrid, Spain , 1 (2006).
- [18] C. Yang, T. Geng, T. Wang, C. Lin, J. Sheng, V. Sachdeva, W. Sherman, and M. Herbordt, Molecular dynamics range-limited force evaluation optimized for FPGAs, *IEEE 30th Int. Conf. Appl. Syst. Archit. Process.* , 263 (2019).
- [19] D. Jones, J. E. Allen, Y. Yang, W. F. Drew Bennett, M. Gokhale, N. Moshiri, and T. S. Rosing, Accelerators for Classical Molecular Dynamics Simulations of Biomolecules, *J. Chem. Theory Comput.* **18**, 4047 (2022).
- [20] OpenMM. <https://openmm.org/benchmarks> (2024).
- [21] E. Luttmann, D. L. Ensign, V. Vaidyanathan, M. Houston, N. Rimón, J. Øland, G. Jayachandran, M. Friedrichs, and V. S. Pande, Accelerating molecular dynamic simulation on the cell processor and playstation 3, *Journal of Computational Chemistry* **30**, 268 (2009).
- [22] I. S. Ufimtsev and T. J. Martinez, Quantum chemistry on graphical processing units. 2. direct self-consistent-field implementation, *Journal of Chemical Theory and Computation* **5**, 1004 (2009).
- [23] H. Lei and Y. Duan, Two-stage Folding of HP-35 from Ab Initio Simulations, *J. Mol. Biol.* **3**, 196 (2007).
- [24] H. Lei, Y. Su, L. Jin, and Y. Duan, Folding Network of Villin Headpiece Subdomain, *Biophys. J.* **99**, 3374 (2010).